## Tutorial Quiz

## Set C

**Instructions: Write the complete function given for the following questions.**

The structure of Listnode for Q1:

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode(int x) : val(x), next(NULL) {}
};
```

The structure of Listnode for Q2 and Q3:

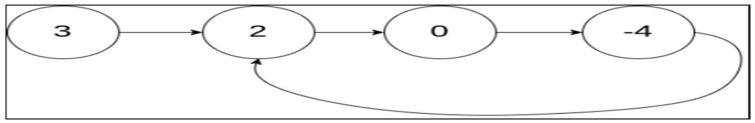
```
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};
```

**Q1.** Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. **Note that pos is not passed as a parameter**.

Return true *if there is a cycle in the linked list*. Otherwise, return false.

**Example 1:**




**Input:** head = [3,2,0,-4], pos = 1

**Output:** true

**Explanation:** There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

Function declaration: **bool hasCycle(ListNode *head) {}**

## Tutorial Quiz

## Set D

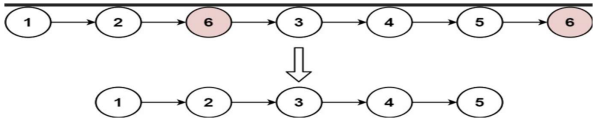
**Instructions: Write the complete function given for the following questions.**

The structure of Listnode for all Questions:

```
struct ListNode {
    int val;
    ListNode *next;
    ListNode() : val(0), next(nullptr) {}
    ListNode(int x) : val(x), next(nullptr) {}
    ListNode(int x, ListNode *next) : val(x), next(next) {}
};
```

**Q1**. Given the head of a linked list and an integer key, remove all the nodes of the linked list that has Node.val == key, and return *the new head*.

**Example 1:**




**Input:** head = [1,2,6,3,4,5,6], val = 6

**Output:** [1,2,3,4,5]

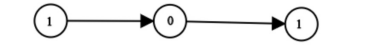
Function declaration: **ListNode* removeElements(ListNode* head, int val) {}**

**Q2.** Given head which is a reference node to a singly-linked list. The value of each node in the linked list is either 0 or 1. The linked list holds the binary representation of a number.

Return the *decimal value* of the number in the linked list.

The **most significant bit** is at the head of the linked list.

**Example 1:**




**Input:** [1,0,1]

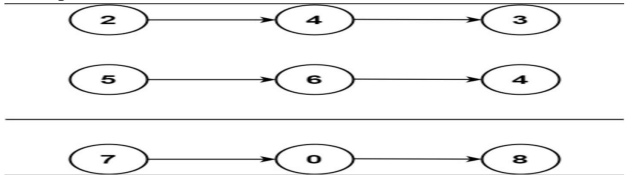
**Output:** 5

**Explanation:** (101) in base 2 = (5) in base 10

Function declaration: **int getDecimalValue(ListNode* head) {}**

**Q3.** You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.
You may assume the two numbers do not contain any leading zero, except the number 0 itself.

**Example 1:**




**Input:** l1 = [2,4,3], l2 = [5,6,4]
**Output:** [7,0,8]
**Explanation:** So the first linked list stores integer 342 in the reversed order as 243 and similarly, the second linked list stores integer 465 in the reversed order as 564, sum of 342 and 465 is 807, and stored in reversed order as 708 in a result linked list.

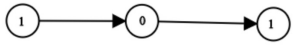
Function declaration: **ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {}**

**Q2.** Given head which is a reference node to a singly-linked list. The value of each node in the linked list is either 0 or 1. The linked list holds the binary representation of a number.

Return the *decimal value* of the number in the linked list.

The **most significant bit** is at the head of the linked list.

**Example 1:**




**Input:** [1,0,1]

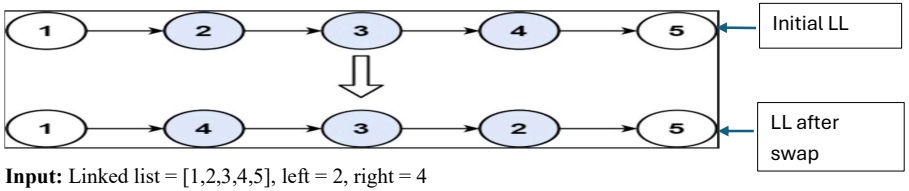
**Output:** 5

**Explanation:** (101) in base 2 = (5) in base 10

Function declaration: **int getDecimalValue(ListNode* head) {}**

**Q3.** Given a link to a node of a linked list of integers where left node value of linked node is less than the right node value, swap the left node value with right node, and return *the reversed list*.

**Example 1:**




**Input:** Linked list = [1,2,3,4,5], left = 2, right = 4

**Output:** [1,4,3,2,5]

Function declaration: **ListNode* reverseBetween(ListNode* head, int left, int right) {}**